

DESIGN DOCUMENT · SELF-HOSTED

Link to Plate: a Discord-native recipe pipeline

A bot that turns a pasted URL into a clean, searchable recipe, lets you slot recipes onto next week's calendar from inside Discord, fills your Woolworths basket for you, and learns when you like your groceries delivered — running entirely on your Unraid box.

Interface Discord slash commands + DM listener **Host** Unraid Docker

Data PostgreSQL **Status** Draft v2

01 What it does

Six jobs, in order: **capture** a recipe from a link or a photo without the life-story preamble and ad blocks, **plan** which recipes cover which day of the coming week — on request, the bot proposes the week itself, weighted by how often you actually want to eat each thing — **consolidate** that week's ingredients into one list, **fill the basket** on your Woolworths account — proposing a substitute from your own reference material wherever something's unavailable — and **book delivery** at a day and time it learns you actually prefer. All six happen without leaving Discord.

TRIGGER

Paste a URL, or attach a photo, in a DM or #recipes channel

STORAGE

Postgres — recipes, parsed ingredients, weekly plan

SHOPPING LIST

Categorized, quantity-merged,

EXTRACTION

schema.org recipe data or Claude vision, into one fixed schema

PLANNING

Tier-weighted suggestions, refined by your like/dislike feedback

CHECKOUT

Basket auto-filled on Woolworths;

substitutions proposed

delivery slot suggested from history

02 The workflow, end to end

1 Drop a link, or a photo

You paste a recipe URL, or attach a photo of a cookbook page or recipe card, into a DM with the bot or a dedicated `#recipes` channel. The bot reacts with an hourglass to confirm it saw it.

2 Bot extracts the recipe

For a link, it looks for embedded `schema.org/Recipe` data first, falling back to Claude on the stripped page text when that's missing. For a photo, it goes straight to Claude's vision input. Either path lands in the same fixed schema — title, ingredients, steps, timing.

3 You confirm the save

The bot posts an embed — title, photo, servings, ingredient count — with // reactions.  stores it as-is,  opens a quick edit (bad extractions happen),  discards it.

4 Recipe lands in your library

Stored with normalized ingredients (quantity, unit, item), searchable with `/recipe search`. Tag it with `/recipe tier` — high, medium, or low rotation, or don't-suggest — so it knows how often you actually want to see it come up.

5 Ask for a plan

`/plan suggest` proposes a full week, weighted toward high-rotation recipes and away from anything you've disliked or eaten recently; swap any day you don't like, or skip this and assign recipes yourself with `/plan set`. `/plan show` prints the week as a Mon–Sun table either way.

6 Generate the shopping list

`/shopping list` pulls every recipe on the week's plan, merges matching ingredients ("2 cups flour" + "1 cup flour" → "3 cups flour"), groups by aisle, and posts it as a checkable list.

7 Round out the list

The bot asks if anything else is needed this week that isn't tied to a recipe — paper towels, dog food — showing your usual answers as quick picks alongside a free-text option.

8 Confirm the list

`/shopping confirm` checks each item against your Woolworths account. Anything out of stock or unmatched gets checked against your uploaded substitution reference, and if there's an accepted swap the bot proposes it inline — "buttermilk unavailable, substitute milk + lemon juice (1 cup + 1 tbsp)?" — with a one-tap approve.

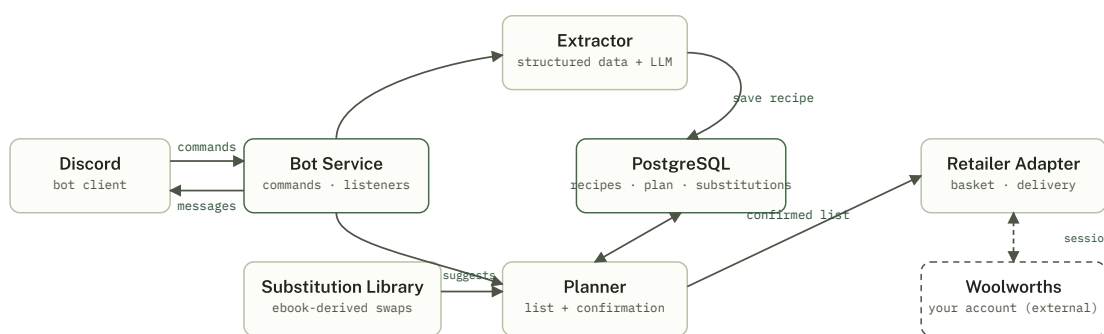
9 Basket fills itself

Once you've cleared the substitution prompts, the bot adds every confirmed item to your Woolworths basket via an authenticated browser session. You still open Woolworths yourself to review and pay — the bot stops at "ready to check out."

10 Pick a delivery slot

The bot DMs the available delivery windows. Once it has enough history, your usual day and time is pre-selected as the top option — accept with one tap, or pick something else for that week.

03 Architecture



The bot service stays a thin translation layer over Discord. The extractor and planner both read and write the same Postgres instance; the substitution library is just

another table in that same database, surfaced to the planner at confirmation time. Only the retailer adapter reaches outside the stack — everything it touches downstream of the planner is a live, authenticated Woolworths session that isn't yours to script casually (more on that in [§09](#)).

04 Data model

RECIPES

COLUMN	TYPE	NOTES
id	uuid	primary key
source_url	text	unique — re-adding a URL updates, doesn't duplicate
title , image_url	text	from extraction
servings , prep_min , cook_min	int	nullable, extractor confidence varies by site
instructions	jsonb	ordered step list
tags	text[]	user-assigned, e.g. weeknight , freezer
rotation_tier	text	high / medium / low / do_not_suggest — drives /plan suggest weighting
added_by , created_at	bigint, timestampz	Discord user id + timestamp

INGREDIENTS

COLUMN	TYPE	NOTES
id	uuid	primary key
recipe_id	uuid	fk → recipes
raw_text	text	original line, kept for display and re-parsing
quantity , unit ,	numeric, text,	parsed fields, used for merging

name	text	
aisle	text	produce / dairy / pantry / meat / etc., for list grouping

PLAN_ENTRIES

COLUMN	TYPE	NOTES
id	uuid	primary key
week_start	date	the Monday of the planned week
day_of_week	smallint	0–6
recipe_id	uuid	fk → recipes
meal_slot	text	default dinner ; open to breakfast/lunch later

RECIPE_FEEDBACK

COLUMN	TYPE	NOTES
recipe_id	uuid	fk → recipes
week_start	date	which week's plan this reaction belongs to
rating	text	liked / disliked , from a reaction on the plan or an explicit /recipe feedback
noted_at	timestamptz	a run of dislikes on one recipe is what triggers a re-tier suggestion

EXTRA_ITEMS

COLUMN	TYPE	NOTES
label	text	free-text item, not tied to any recipe
week_start	date	which week's list it was added to
added_at	timestamptz	"favorites" are just the most frequent labels here — no separate table to keep in sync

SUBSTITUTIONS

SUBSTITUTIONS

COLUMN	TYPE	NOTES
id	uuid	primary key
source_ingredient	text	normalized name being replaced
substitute_ingredient , ratio_note	text	e.g. "milk + lemon juice", "1 cup + 1 tbsp per cup buttermilk"
source_document	text	which uploaded reference this came from
confidence	text	extracted or user_confirmed — flags ambiguous parses for review

RETAILER_ITEM_MAPPINGS

COLUMN	TYPE	NOTES
ingredient_name	text	normalized shopping-list item
retailer_product_id , retailer_product_name	text	the Woolworths match you confirmed once
confirmed_by_user	boolean	false until you approve the first match; auto-applied after
last_used_at	timestamptz	lets stale mappings be re-checked periodically

RETAILER_SESSION

COLUMN	TYPE	NOTES
session_blob	bytea, encrypted	authenticated session/cookie state — never a plaintext password
expires_at , last_refreshed_at	timestamptz	drives the "please re-authenticate" DM when stale

DELIVERY_CHOICES

COLUMN	TYPE	NOTES
order ref	text	whatever the retailer's basket/order flow

returns		
day_of_week , time_window	smallint, text	the slot you picked
chosen_at	timestamptz	most-frequent day/time over recent rows becomes next week's suggestion

05 Discord command surface

COMMAND	DOES
/recipe add url:	Also triggered passively by pasting a link — same path either way
/recipe list [tag]	Paginated embed of saved recipes, optionally filtered by tag
/recipe show name:	Full recipe — ingredients and steps
/recipe search query:	Search by title or ingredient
/recipe delete name:	Removes from the library
/recipe tier name: tier:	Sets rotation tier — high / medium / low / don't-suggest
/recipe feedback name: rating:	Records a like/dislike outside the plan-recap reactions
/plan suggest	Proposes a full week, weighted by tier and feedback history
/plan set day: recipe:	Assigns a recipe to a day of the upcoming week
/plan show	This week's plan as a table, blanks visible
/plan clear day:	Unassigns a day
/shopping list	Builds and posts the merged, aisle-grouped list for the current plan

<code>/shopping extras</code>	Prompts for non-recipe items, showing your usual favorites as quick picks
<code>/shopping confirm</code>	Checks stock, proposes substitutions for anything unavailable, waits for your approval
<code>/shopping send-to-basket</code>	Pushes the confirmed list into your Woolworths basket
<code>/shopping delivery</code>	Prompts for a delivery slot, pre-filled with your usual choice once there's history
<code>/retailer connect</code>	One-time interactive login that establishes the Woolworths session
<code>/substitutions upload</code>	Attach a PDF/EPUB of acceptable substitutions to ingest
<code>/substitutions list</code> ingredient:	Shows what's on file for a given ingredient

06 Recipe capture — links & photos

Two ways in: paste a link, or attach a photo of a page or card. Both end up in the same fixed schema — title, ingredients, steps, timing — before they reach the confirm-embed in §02, so nothing downstream needs to know which path a recipe came in through.

From a link

Most recipe sites already embed machine-readable data for Google's recipe rich-results — that's the cheap, reliable path, and it should handle the large majority of links with no model call at all. Reach for an LLM only when that data is missing or the page actively resists scraping.

TIER	METHOD	COVERS
1	Parse embedded <code>schema.org/Recipe</code> JSON-LD (via <code>recipe-scrappers</code> , which already has per-site handling for hundreds of cooking sites)	Most established recipe blogs and publishers

2	Strip boilerplate from raw HTML (readability-style extraction), then have Claude extract title/ingredients/steps/servings into a fixed schema	Sites without structured data, forum posts, screenshots-as-text situations
3	Manual entry via <code>/recipe add</code> with no URL — bot prompts for fields directly	Sites that block fetching entirely, or a recipe with no link at all

WHY NOT LLM-ONLY

Running every link through a model call works, but it's slower, costs more, and structured data is already exact where it exists. Tiering keeps the common case fast and free, and reserves the model for pages that actually need judgment.

From a photo

Attach an image with no URL — a cookbook page, a handwritten card, a screenshot — and the bot routes it straight to Claude's vision input rather than running a separate OCR pass first: reading the page and structuring it happen in one step, and it holds up better on handwriting and skewed photos than a two-stage OCR-then-parse pipeline would. A genuinely unreadable photo gets a "can you retake that" reply rather than a guessed-at recipe; minor slips are still caught by the same 🖋️ edit reaction as any other capture path.

07 Recipe rotation & weekly suggestions

`/recipe tier` is the primary lever: every recipe is `high`, `medium`, `low`, or `do_not_suggest`, set by you. `/plan suggest` fills the week with a weighted random draw across your library — high rotation drawn far more often than low, `do_not_suggest` excluded outright — with a recency penalty so the same three dinners don't reappear every other week.

Feedback is a secondary signal on top of that, never a silent override. React 👍/👎 on the plan-recap message, or use `/recipe feedback` any time, and both feed `recipe_feedback`. A recipe that racks up repeated dislikes gets weighted down in future draws — and once it crosses a threshold, the bot proposes moving it to a lower tier rather than doing so on its own, the same "suggest, don't decide" pattern used for ingredient substitutions in §09.

Any single day in a suggested week can be swapped without regenerating the rest — reject one recipe and the bot redraws just that slot.

08 Shopping list generation

- **Merge** — ingredients with the same normalized name and compatible units are summed (2 cups flour + 1 cup flour → 3 cups flour). Incompatible units (2 cloves garlic + 1 tsp garlic powder) list separately rather than guessing a conversion.
- **Group** — merged items are bucketed by `aisle` so the list reads in shopping order, not recipe order.
- **Round out** — `/shopping extras` asks what's needed this week outside any recipe. Your most-frequently-added items surface as one-tap picks, with a free-text option for anything new; the pick list is just a frequency query over `extra_items` , not a hand-maintained favorites list.
- **Present** — posted as a Discord message with checkbox reactions per item, so it stays usable as a live list while you're actually in the store.
- **Pantry filter (optional)** — a personal "always have" tag (salt, oil, flour) that can be excluded from the printed list on request.

09 Retailer integration — basket & delivery

Woolworths doesn't offer a public API for personal basket automation, so this component drives the site the same way you would: a headless browser (Playwright) controlling an already-authenticated session, searching for each confirmed item, and adding the best match to the basket.

Authenticating without storing a password

`/retailer connect` opens a one-time interactive login — you sign in yourself, the adapter captures the resulting session, and only that session (not your username or password) is kept, encrypted at rest. When it expires, the bot DMs you to reconnect rather than silently retrying credentials it doesn't have.

Matching items to real products

The first time an ingredient needs a Woolworths match, the bot searches the catalog and asks you to confirm the right product via `/shopping confirm`. That choice is remembered in `retailer_item_mappings`, so most weeks the same ingredient resolves automatically with no prompt at all.

Delivery scheduling

After the basket is filled, `/shopping delivery` shows the available slots. Each choice is logged; once there's enough history the most frequent day and time is offered first, one tap to accept.

HARD BOUNDARIES ON THIS COMPONENT

Basket only, never checkout — payment stays a deliberate action you take yourself in the Woolworths app. If the site throws a CAPTCHA or any other bot challenge, the adapter stops and DMs you to clear it manually; it should never try to defeat one. And automating a retail account like this sits outside most retailers' terms of service in the fine print — that's a real risk to weigh (account flags, at minimum) before this component runs unattended, not a detail to skip past.

10 Ingredient substitution library

You upload a reference — a PDF or EPUB listing acceptable ingredient swaps — once, via `/substitutions upload`. The bot extracts its text and asks Claude to structure it into `source_ingredient` → `substitute_ingredient` rows with a ratio note, the same structured-extraction pattern used for tier-2 recipes. Anything ambiguous is flagged `extracted` rather than `user_confirmed`, and `/substitutions list` lets you review or correct it.

At runtime, this only surfaces during `/shopping confirm`: when the retailer adapter reports an item as out of stock or unmatched, the planner checks this table before bothering you, and only proposes a swap when one is actually on file. No match, no substitution library entry — the item is simply flagged back to you in the confirmation summary instead of guessed at.

11 Stack recommendation

LAYER	CHOICE	WHY
Language	Python	<code>recipe-scrappers</code> and ingredient-parsing libraries are mature here; no reason to fight the ecosystem
Bot framework	<code>discord.py</code> (<code>app_commands</code>)	Native slash-command support, active maintenance, good docs for embeds/reactions
Database	PostgreSQL	Handles jsonb instructions and array tags cleanly; trivial to run as its own Unraid container with a persistent volume
Extraction	Claude API (text + vision)	Structured extraction into a fixed JSON schema — text fallback for links without structured data, vision for photo capture, text again for the substitution ebook
Retailer automation	Playwright	Drives an authenticated browser session; easier to pause for human-in-the-loop steps (login, CAPTCHA) than a reverse-engineered API client, and more transparent when the site changes
Ebook parsing	PyMuPDF / ebooklib	Text extraction for PDF and EPUB substitution references
Deployment	Docker Compose, two containers	Bot + Postgres, managed from Unraid's Docker tab like any other app

ISOLATE THE FRAGILE PART

There's no official Woolworths integration to build against, so the retailer adapter is the one component genuinely likely to break when the site changes layout. Keep it behind its own service boundary with a narrow interface (search, add-to-basket, list slots, book slot) so a Woolworths redesign is a contained fix, not a rewrite of the planner or the bot.

Suggested repo layout

```
recipe-bot/  
  bot/  
    main.py  
    commands/      recipe.py · plan.py · shopping.py · retailer.py  
    listeners/      url_listener.py  
  scraper/  
    extractor.py    tier 1 + tier 2 dispatch  
    llm_extractor.py Claude fallback  
    normalizer.py   ingredient → quantity/unit/name  
  substitutions/  
    ingest.py       ebook → structured substitution rows  
  retailer/  
    adapter.py      search · add-to-basket · list/book slots  
    session.py      login capture, encrypted storage, refresh  
  services/  
    recipe_service.py  
    plan_service.py  
    shopping_service.py  
  db/  
    models.py  
    migrations/  
  docker-compose.yml  
  Dockerfile  
  .env.example
```

Minimal docker-compose sketch

```
services:  
  bot:  
    build: .  
    restart: unless-stopped  
    env_file: .env  
    depends_on: [db]  
  db:  
    image: postgres:16  
    restart: unless-stopped  
    environment:  
      POSTGRES_DB: recipes  
      POSTGRES_USER: recipes  
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password  
    volumes:  
      - /mnt/user/appdata/recipe-bot/pgdata:/var/lib/postgresql/data
```

12 Open decisions

Which Woolworths storefront?

The adapter's selectors and slot/catalog structure are storefront-specific (AU vs NZ, for instance). **Recommendation:** confirm which one before Phase 5 — it's a small detail now and a rewrite later.

What happens when there's no stock match and no substitution on file?

Recommendation: leave it off the basket and list it plainly in the confirmation summary rather than guessing — a missed item you notice beats a wrong one you don't.

How aggressive should ingredient merging be?

Unit conversion (cups ↔ tbsp ↔ oz) adds real complexity. **Recommendation:** start with exact-unit merging only, ship it, and add conversion once you see how often it actually matters in practice.

Should checkout ever be automated?

Recommendation: no — keep this a firm boundary, not just a default. Basket population is a convenience; payment is a decision you make each time you see the final total.

Where's the dislike threshold for a re-tier prompt?

Recommendation: start at three dislikes with no likes in between, then tune once you see how often real feedback triggers it — too eager and it nags, too slow and a recipe you've soured on keeps getting suggested.

13 Phased build plan

PHASE 1

PHASE 2

PHASE 3